

Sintesi e Fondamenti di Assembly x86-64 (Sintassi Intel)

NECKER

14 Novembre 2025

Indice

1	Introduzione: Sintassi Intel vs AT&T	2
1.1	Convenzione degli Operandi (Sintassi Intel)	2
2	Tipi di Registri e Convenzioni d'Uso (x86-64)	2
2.1	Registri General Purpose (GPR)	2
2.2	Registri di Controllo dello Stack	3
3	Direttive Assembler per la Riserva di Spazio	3
3.1	Direttive di Sezione	3
3.2	Direttive di Definizione Dati	3
3.2.1	Riserva di Spazio (Non Inizializzato)	3
4	Istruzioni Assembly Fondamentali	4
4.1	Movimento Dati	4
4.2	Aritmetica e Logica	4
4.3	Controllo del Flusso	4
5	Interfacciamento con il Sistema Operativo (Syscalls)	5
5.1	Convenzione Linux/System V x86-64	5
5.2	Esempio: Syscall <code>exit</code> (uscita dal programma)	5

1 Introduzione: Sintassi Intel vs AT&T

Il linguaggio Assembly è un linguaggio di programmazione a basso livello che corrisponde quasi direttamente al linguaggio macchina. L'architettura **x86** (e successiva **x86-64**) supporta due sintassi principali: **Intel** e **AT&T**.

1.1 Convenzione degli Operandi (Sintassi Intel)

La sintassi Intel è caratterizzata dall'ordine degli operandi, che segue la logica:

Istruzione Destinazione, Sorgente

L'operazione è concettualmente un'assegnazione: il contenuto della **Sorgente** viene spostato nella **Destinazione**.

```
1 ; Intel: dest, source
2 mov rax, rbx ; Copia il contenuto di RBX in RAX
3 mov rax, [rcx] ; Copia il valore dalla memoria puntata da RCX
4 in RAX
```

Listing 1: Esempio di Sintassi Intel

2 Tipi di Registri e Convenzioni d'Uso (x86-64)

L'architettura x86-64 estende i registri a 64 bit (prefisso 'R') e aumenta il loro numero. L'uso dei registri segue precise **convenzioni di chiamata** (*calling conventions*) stabilite dal sistema operativo (es. System V ABI per Linux, o Microsoft x64).

2.1 Registri General Purpose (GPR)

I registri sono ampi 64 bit, ma possono essere acceduti anche in sezioni minori (EAX per 32 bit, AX per 16 bit, AL/AH per 8 bit).

- **RAX (Accumulatore):** Utilizzato per l'aritmetica e i valori di **ritorno** delle funzioni. Nelle **syscall**, contiene il numero della funzione di sistema.
- **RBX (Base):** Registro **callee-saved** (il chiamato deve salvarlo nello Stack). Uso generale.
- **RCX, RDX, R8, R9:** Utilizzati principalmente per passare i primi quattro **argomenti** alle funzioni (in Linux/Windows, l'ordine preciso varia). Nelle **syscall**, fungono da argomenti.
- **RSI (Source Index):** Puntatore alla sorgente per operazioni su stringhe/blocchi.
- **RDI (Destination Index):** Puntatore alla destinazione per operazioni su stringhe/blocchi. Il primo **argomento** nelle convenzioni di chiamata System V ABI (Linux).

2.2 Registri di Controllo dello Stack

- **RSP (Stack Pointer):** Punta sempre alla **cima** (top) corrente dello Stack. **Mobile** e usato da PUSH / POP.
- **RBP (Base Pointer):** Usato come **Frame Pointer**. Indica l'inizio **fisso** del Record di Attivazione (Stack Frame) di una funzione, facilitando l'accesso a variabili locali e argomenti. È *callee-saved*.

3 Direttive Assembler per la Riserva di Spazio

Le **direttive** sono istruzioni per l'assemblatore (non per la CPU) che servono a definire sezioni, allocare variabili e riservare spazio in memoria, in particolare nel segmento dati (`.data`) o nel segmento di Stack (`.stack`).

3.1 Direttive di Sezione

- `.data` o `section .data`: Definisce il segmento in cui allocare variabili e dati inizializzati, cioè già definiti all'inizio del programma (stringhe di output predefinite).
- `.bss` o `section .bss`: Definisce il segmento per variabili e dati **non inizializzati** (variabili che esisteranno).
- `.text` o `section .text`: Definisce il segmento per il codice eseguibile (istruzioni).

3.2 Direttive di Definizione Dati

Queste direttive riservano spazio e, opzionalmente, lo inizializzano. Il formato generale è: `Nome_Variabile Direttiva Valore(i)`.

Direttiva	Nome Completo	Dimensione
DB	Define Byte	1 Byte
DW	Define Word	2 Byte (16 bit)
DD	Define Doubleword	4 Byte (32 bit)
DQ	Define Quadword	8 Byte (64 bit)

3.2.1 Riserva di Spazio (Non Inizializzato)

Per riservare spazio senza inizializzare (tipicamente in `.bss`), si usano direttive come `RESB`, `RESW`, `RESQ`, con l'argomento che indica il **numero di unità** da riservare.

```
1  section .data
2  msg DB 'Hello', 0 ; Variabile 'msg': 6 byte inizializzati (
   stringa + terminatore nullo)
3
4  section .bss
```

```

5  buffer RESB 1024      ; Variabile 'buffer': Riserva 1024 byte
6  contatore RESQ 1     ; Variabile 'contatore': Riserva 8 byte (un
7  quadword)

```

Listing 2: Esempi di Riserva Spazio

4 Istruzioni Assembly Fondamentali

Le istruzioni sono i comandi eseguiti dalla CPU.

4.1 Movimento Dati

- `MOV dest, src`: Sposta dati tra registri, tra registro e memoria, o tra valore immediato e registro/memoria.
- `PUSH src`: Sposta l'operando nello Stack e decrementa `RSP`.
- `POP dest`: Estrae l'operando dalla cima dello Stack e incrementa `RSP`.

4.2 Aritmetica e Logica

- `ADD dest, src`: $dest = dest + src$.
- `SUB dest, src`: $dest = dest - src$.
- `MUL src`: Moltiplica `src` per il registro **RAX**. Il risultato a 128 bit è in **RDX:RAX**.
- `INC dest`: $dest = dest + 1$.
- `DEC dest`: $dest = dest - 1$.
- `AND`, `OR`, `XOR`, `NOT`: Operazioni logiche a livello di bit.

4.3 Controllo del Flusso

- `CMP op1, op2`: Confronta gli operandi (`op1 - op2`) e imposta i flag di stato (ZF, CF, SF) senza modificare gli operandi.
- `JMP label`: Salto incondizionato all'etichetta.
- `Jcond label`: Salto condizionato basato sui flag (es. `JE` per salto se uguale/Zero Flag settato, `JG` per salto se maggiore, ecc.).
- `CALL label`: Salva l'indirizzo di ritorno nello Stack ed esegue un salto.
- `RET`: Estrae l'indirizzo di ritorno dallo Stack e salta a tale indirizzo.

5 Interfacciamento con il Sistema Operativo (Syscalls)

Le **System Calls** (`syscall`) sono il meccanismo con cui un programma Assembly richiede servizi al kernel del sistema operativo (es. I/O, creazione processi, allocazione memoria).

5.1 Convenzione Linux/System V x86-64

L'esecuzione di una `syscall` segue i seguenti passi (Linux):

1. Caricare il **numero della syscall** nel registro **RAX**.
2. Caricare gli **argomenti** della syscall in registri specifici:
 - Argomento 1: **RDI**
 - Argomento 2: **RSI**
 - Argomento 3: **RDX**
 - Argomento 4: **R10**
 - Argomento 5: **R8**
 - Argomento 6: **R9**
3. Eseguire l'istruzione `syscall`.
4. Il valore di **ritorno** della syscall viene scritto in **RAX**.

5.2 Esempio: Syscall exit (uscita dal programma)

```
1  section .text
2  global _start
3
4  _start:
5  ; 1. Codice della syscall (exit = 60) in RAX
6  mov rax, 60
7  ; 2. Argomento 1 (codice di uscita) in RDI
8  mov rdi, 0      ; Exit code 0 (successo)
9  ; 3. Esegue la syscall (che e' contenuta nel registro RAX)
10 syscall        ; Termina il programma
11
```

Listing 3: Esempio di Syscall per Uscire (exit)